

NAT and port forwarding at the edge

Follow-along: make a private site reach out, then make one service in it reachable

Overview

Definition: Network address translation

A router rewriting addresses, and usually port numbers, in the header of a packet it forwards, and applying the exact reverse rewrite to the packets of the same connection coming back. Neither end host is changed and neither is told: each keeps addressing the address it started with. RFC 3022 calls the variant that rewrites the address alone Basic NAT, and the variant that rewrites the address and the transport identifier as well (a TCP or UDP port, or an ICMP query identifier) NAPT. Every rule in this lab is NAPT.

A site in this lab is three machines on 192.168.10.0/24, a prefix out of the space RFC 1918 reserves for private use. Anybody may number a network out of it without asking a registry, which is why it is used, and routing information about it is not carried between enterprises, which is the price. The one machine on the far side of the edge router holds no route back to 192.168.10.0/24 and there is nothing the site can do about that, because the far side is not the site's to configure. So a private client's request arrives and its reply is never sent. **Network address translation** on the edge router is what closes that gap, by putting an address in the packet that the far side can reply to.

The lab builds three translations and each one fails differently when it is absent. A source rewrite on the way out gives the far side an address to reply to. A destination rewrite on the way in makes one service inside the site reachable through the router's own address, which traditional NAT treats as an exception to its outbound-only model rather than as a second mode. A second source rewrite handles the case a working port forward breaks: a private client asking for the site's own public address, which a server three metres away replies to directly, and which the client then rejects. Left out, because none of them changes any result here: IPv6, carrier-grade NAT and the address-sharing ratios it is sized by, NAT as a packet filter, and the protocols that carry addresses inside their payload and need a connection-tracking helper to work at all.

Running this lab in the TUI

You drive the lab from the MiniLabs launcher. Clone it if the machine does not have it, and pull if it does, because handouts and lab scripts change together:

```
git clone https://github.com/nzRichie/MiniLab.git ~/MiniLab # first time
cd ~/MiniLab && git pull # already cloned
```

Start the launcher from that directory:

```
cd ~/MiniLab && ./minilabs
```

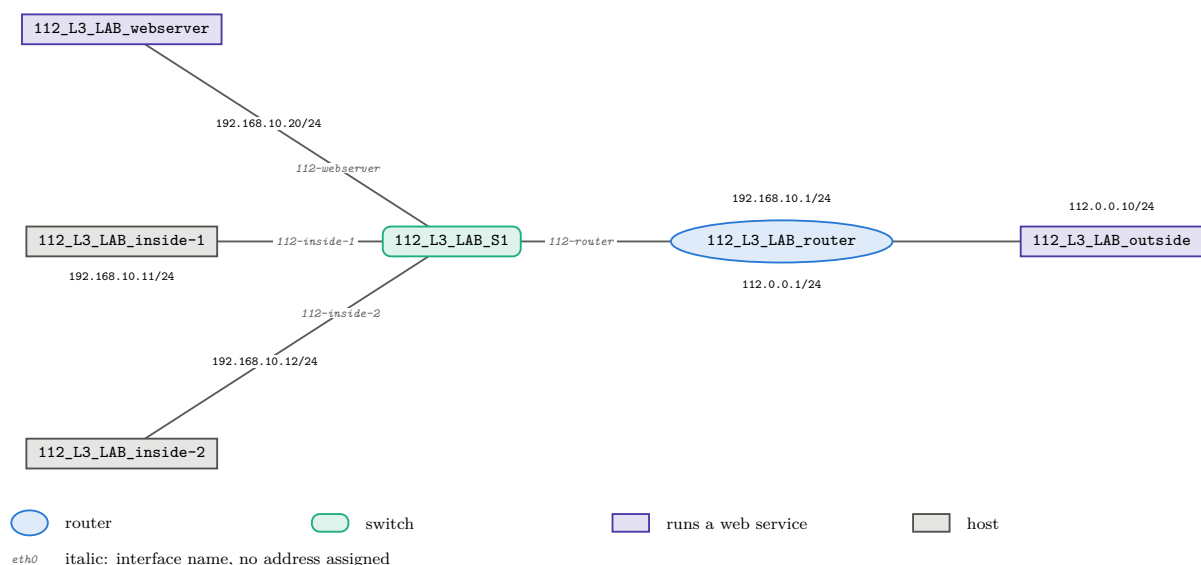
If it exits with a Docker error, follow the setup in the repository's README.md and run it again.

The root menu lists one category per row. Select **Network configuration**, then **NAT and port forwarding at the edge**, to reach this lab's actions:

- **Spawn** creates this lab's containers and wiring. Wait for the panel to turn green before you go on. The first spawn of a lab also builds its image; later spawns take seconds.
- **Status** prints the lab's state and its success oracle, so you read the before and after without shelling in. It makes the same three requests you make by hand, and for each one it reports whether it arrived and what source address the machine at the far end saw. Status takes over the whole screen: the arrow keys scroll its output, and Enter on **Done** closes it.
- **Shell command for a node** lists every node (router, inside-1, inside-2, webserver, outside) and the `docker exec -it ...` line that opens a shell on it. Copy the line for the node a step below names into your own terminal. It holds the screen until Enter on **Done**, as Status does.
- **Reset to baseline** deletes every rule you wrote on the edge router and puts both web services back, without tearing the lab down.
- **Teardown** removes the lab.
- **Open handout** opens this document.

Spawn the lab now, then work through Part 1 in your own terminal. **Back** (or Esc) returns to the lab list.

The setup



Six containers, two segments, one router between them. The private segment is switched because three machines share it; the outside link carries two machines and has no switch on it. Nothing filters anywhere: the router forwards every packet it is handed, so a packet that does not arrive was never sent or was never addressed to somewhere reachable, and no rule dropped it.

Every container is named `112_L3_LAB_` followed by the name in the first column, so the edge router is `112_L3_LAB_router`.

Device	Interface	Address	Runs
router	inside	192.168.10.1	forwarding; nft and conntrack
	outside	112.0.0.1	
inside-1	112-S1	192.168.10.11	curl , ping , tcpdump
inside-2	112-S1	192.168.10.12	the same
webserver	112-S1	192.168.10.20	lighttpd on tcp/80
outside	112-ext	112.0.0.10	lighttpd on tcp/80
S1	112-inside-1, 112-inside-2, 112-webserver, 112-router	none	one Open vSwitch bridge

The three private machines have 192.168.10.1 as their default gateway. The outside host has no default route at all, and the only route it holds is the one for its own segment. That is not an omission to be corrected: it is what a host beyond your site looks like, and it is the condition every part of this lab is measured against.

Both web services serve two things. A request for `/` returns a marker naming which service replied, `OUTSIDE-SITE-3D18B4` or `INSIDE-SITE-9C40FA`, so a fetch that succeeds says which machine it reached. A request for `/whoami.cgi` returns the source address of the connection the server is replying to, one address on one line. That second endpoint is what almost every question below turns on, because the source address a server sees is the single thing each of the three translations changes.

Four commands do the work:

- **nft** loads and lists the router's rules.
- **conntrack -L** prints the router's connection-tracking table, and **conntrack -F** empties it.
- **curl** makes the requests. **curl -v** is worth the extra output here: the summary line collapses several different failures into one sentence and the verbose line does not.
- **tcpdump -n -i <interface> <filter>** captures on one interface. **-n** prints addresses rather than looking up names for them, and the filter is an expression that selects which packets are shown: a filter of `tcp port 8080` selects packets carrying 8080 as their source or their destination port, and the blocks below show how each filter is quoted for the shell.

Assessment

There are sixteen questions, placed at the step that produces the observation each one asks about. Answer all sixteen in a separate document, numbered to match. Several ask you to record a value your own machine printed; record what it printed, not what you expected.

Part 1: the site before any translation

Spawn the lab and open a shell on **inside-1**. The private segment works from the first second:

```
# 112_L3_LAB_inside-1: a private client
curl http://192.168.10.20/
away
```

```
# the private web service, one hop \
```

Now aim at the far side.

```
# 112_L3_LAB_inside-1
ping -c 3 112.0.0.10
curl --max-time 8 http://112.0.0.10/
```

Both fail, and neither says why. The router is forwarding: it has an address on both segments and `ip_forward` is on, which you can confirm but not change, because Docker mounts `/proc/sys` read-only in a container that is not privileged and this lab does not make the router privileged. So the requests are leaving. Find out whether they arrive. Open a second shell on the outside host, start a capture, and run the ping again from the first shell:

```
# 112_L3_LAB_outside: the host beyond the edge
tcpdump -n -i 112-ext icmp
```

```
# 112_L3_LAB_inside-1: in the shell you already have open
ping -c 2 112.0.0.10
```

```
# 112_L3_LAB_outside prints this; it is output, not commands to type
IP 192.168.10.11 > 112.0.0.10: ICMP echo request, id 53183, seq 1, length 64
IP 192.168.10.11 > 112.0.0.10: ICMP echo request, id 53183, seq 2, length 64
```

The requests arrive, carrying the private client's own address, and nothing goes back. Ask the outside host's routing table what it would do with a reply:

```
# 112_L3_LAB_outside
ip route get 192.168.10.11
ip route
```

Question 1. `ip route get 192.168.10.11` on the outside host prints one line. Quote it, then explain what stopped the echo reply and name the machine the reply was lost on. Your answer should say whether the reply was ever put on a wire.

Question 2. Name the one change to the outside host that would let `inside-1` fetch `http://112.0.0.10/` with no translation configured anywhere. Then say why that change is not available to a site whose far side is the internet rather than one container you happen to control. Your second half should name a property of `192.168.10.0/24` specifically, not of the container.

Part 2: giving the far side an address it can reply to

Definition: Source NAT

Rewriting the source address, and usually the source port, of a packet on its way out, so that replies are addressed to the router instead of to the sender. It is performed at the `postrouting` hook, after the routing decision has chosen an outgoing interface, because until that decision is made the router does not know which of its addresses the packet should appear to come from.

Definition: Table, chain, hook and priority in nftables

A *table* is a named container for chains, tied to a protocol family (`ip` for IPv4). A *base chain* is a chain given a *hook*, a named point in the kernel's packet path, and a *priority*, a number deciding the order among chains on the same hook. A chain without both is a regular chain, reachable only as a jump target, and it sees no packets at all. Only a base chain carries a *policy*, the verdict for a packet that reached the end of the chain with no rule deciding its fate; `accept` is the default. Standard priorities have names, and the numbers behind them depend on the family: in the `ip` family used here `dstnat` is `-100` and belongs to `prerouting`, and `srcnat` is `100` and belongs to `postrouting`. Chains are traversed in ascending order, which is what puts a destination rewrite before the routing decision and a source rewrite after it.

The router needs **source NAT** on traffic leaving by its outside interface. That is three commands: a **table** to hold the work, a base chain hooked at `postrouting`, and one rule. Nothing exists yet, so all three are creations rather than edits.

```
# 112_L3_LAB_router: the edge router, and the only machine you configure
nft add table ip nat
nft add chain ip nat postrouting '{type nat hook postrouting priority srcnat;}'
nft add rule ip nat postrouting oifname "outside" ip saddr 192.168.10.0/24 masquerade
nft list ruleset
```

Definition: masquerade

An nftables statement that performs source NAT to the address of the interface the packet is leaving by, rather than to an address written in the rule. It is valid only in a chain of type `nat`, and only at `postrouting`, because the outgoing interface is what it reads the replacement address from. Its sibling `snat` to `112.0.0.1` names the address instead. A masquerading rule also has the kernel forget a connection when the interface it was translated onto goes down.

The rule reads as three conditions and one action. `oifname "outside"` restricts it to packets the routing decision is sending out of the outside interface, so nothing on the private segment is touched. `ip saddr 192.168.10.0/24` restricts it to packets from the site's own prefix. **masquerade** is the action, and it takes the replacement address from the outgoing interface rather than from anything written in the rule.

Now repeat Part 1's two commands from `inside-1`, then ask the outside server what it saw:

```
# 112_L3_LAB_inside-1
ping -c 3 112.0.0.10
curl http://112.0.0.10/
curl http://112.0.0.10/whoami.cgi
```

Definition: Connection tracking

The kernel subsystem that records every connection crossing the machine as a pair of tuples: the *original* tuple, which is what the machine that opened the connection sent, and the *reply* tuple, which is what the router expects a reply to look like. A NAT rule is consulted once, for the first packet of a connection, and what it decides is stored in that entry; every later packet of the same connection is translated from the entry and never looks at the ruleset again. Loading a rule therefore does nothing to connections already being tracked, and `conntrack -F` empties the table so the next packet starts a connection the current ruleset gets to decide.

One rewrite happened, and **connection tracking** is where the router wrote it down. Read the entry that fetch created:

```
# 112_L3_LAB_router
conntrack -L -p tcp
```

Each line holds eight address and port fields, in two groups of four, and the layout is what has to be read before the values mean anything:

```
# 112_L3_LAB_router prints one line per flow in this shape; it is not a command to \
type
tcp 6 119 TIME_WAIT src=A dst=B sport=a dport=b src=C dst=D sport=c dport=d [ASSURED]
```

`src=A dst=B sport=a dport=b` is the original tuple: what the machine that opened the connection put in its header. `src=C dst=D sport=c dport=d` is the reply tuple: what the router will recognise a reply by. On a router translating nothing the two are one exchange apart, with `C` equal to `B`, `D` equal to `A`, `c` equal to `b` and `d` equal to `a`, because a reply to a packet goes back to whoever sent it. Read your own line against that and find every place it does not hold.

Question 3. Give the address `/whoami.cgi` returned to `inside-1`. Then, from your own `conntrack` entry for that fetch, give the value of `dst=` in the reply tuple, and state what the router does with an arriving packet whose header matches that reply tuple.

Question 4. The reply tuple is not the original tuple with its source and destination exchanged. Name the single field where it differs from that exchange, and name the rule you loaded that produced the difference.

Try the same reading for ICMP, which has no ports at all:

```
# 112_L3_LAB_inside-1
ping -c 2 112.0.0.10
```

```
# 112_L3_LAB_router
conntrack -L -p icmp
```

Question 5. In the ICMP entry, name the field that appears where a TCP entry carries `sport` and `dport`. Then say what the router needs that field for, given that both private clients would otherwise be translated to the same single address.

A masquerading rule leaves the source port alone when the port is free, so two clients picking their own ports at random almost never collide and you would not see the port half of NATP working. Force a collision. Open a shell on each private client and pin both to the same source port. The `nc` command connects and then waits, so leave each one running and open the next shell:

```
# 112_L3_LAB_inside-1: leave this running
nc -p 40000 112.0.0.10 80
```

```
# 112_L3_LAB_inside-2: leave this running too
nc -p 40000 112.0.0.10 80
```

```
# 112_L3_LAB_router
conntrack -L -p tcp --dport 80
```

Question 6. Both clients sent from source port 40000. Give the reply-direction `dport` the router recorded for each of the two flows, say which of the two flows was rewritten, and explain what the router would be unable to do if it had left both flows alone.

Question 7. `masquerade` takes the source address from the outgoing interface; `snat to 112.0.0.1` is written with the address in the rule. State one thing each gives you that the other does not, then say which of the two you would put on this router if its outside address were handed to it by a provider and changed from time to time, and why.

Stop both `nc` commands with Ctrl-C before going on.

Part 3: nothing reaches in

Source NAT is one direction. Traditional NAT treats sessions as outbound from the private network, and an inbound one as an exception arranged in advance rather than something the mechanism supports by itself. Test that from the outside host. Two targets: the private server's own address, and the router's outside address on the port the site would like to publish the service on.

Question 8. Before running either command, predict what the outside host will report for each of the two fetches below, and give a reason for each prediction. Then run them and record the reason line `curl -v` printed. If a prediction was wrong, say so; the mark is for the reasoning either way.

```
# 112_L3_LAB_outside: the host beyond the edge
curl -v --max-time 5 http://192.168.10.20/
curl -v --max-time 5 http://112.0.0.1:8080/
```

Both fetches end with curl exit status 7 and the same summary line, `Could not connect to server`. The verbose output distinguishes them and the summary does not.

Question 9. Name what happened to the packet in each of the two fetches, and say which of the two put a packet on the wire at all. For the one that did, name the machine that replied to it and what it sent back.

Part 4: publishing one service through the router's address

Definition: Destination NAT

Rewriting the destination address, and usually the destination port, of a packet on its way in, so that a packet addressed to the router is delivered to a machine behind it. It is performed at the **prerouting** hook, before the routing decision, because the decision has to be made on the new destination and not the old one. Rewriting a destination port as well as an address is what is usually meant by a *port forward*.

The site wants its private web server reachable from outside on 112.0.0.1 port 8080. **Destination NAT** at **prerouting** does it, and the chain is a new one, because **prerouting** is a different hook from the one Part 2 registered:

```
# 112_L3_LAB_router
nft add chain ip nat prerouting '{type nat hook prerouting priority dstnat;}'
nft add rule ip nat prerouting iifname "outside" tcp dport 8080 dnat to \
    192.168.10.20:80
nft list ruleset
```

The published port and the service's own port are independent values, and here they differ on purpose: the rule matches destination port 8080 and hands the packet to port 80 on 192.168.10.20.

```
# 112_L3_LAB_outside
curl http://112.0.0.1:8080/
curl http://112.0.0.1:8080/whoami.cgi
```

```
# 112_L3_LAB_router
conntrack -L -p tcp --dport 8080
```

Question 10. Give the address `/whoami.cgi` returned to the outside host through the published port, and put it beside the address you recorded for the equivalent request in Question 3. Explain, in terms of which header field each of the two rules rewrites, why the two are not the same.

Question 11. The outside host sent its request to 112.0.0.1 port 8080. Your **conntrack** reply tuple says the reply will arrive from 192.168.10.20 port 80. Explain why the outside host accepts a reply at all, given that it opened its connection to neither of those two values.

Question 12. Three packets carry a destination or source port that is either 8080 or 80: the packet the outside host sends, the packet the private web server receives, and the reply the outside host reads. Give the value each of the three carries, and say which device changed it and at which hook.

Part 5: the same address, asked for from inside the site

A port forward that works from outside is not finished. Someone on the private segment will use the same published address, because it is the address the service is advertised at, and a private client is the one machine on which that address behaves differently. Try it.

Question 13. The rule you loaded in Part 4 begins `iifname "outside"`. Predict what `inside-1` will get from `curl -v http://112.0.0.1:8080/` while that rule is loaded, name the machine that will produce that reply, and say why the rule has nothing to do with it.

```
# 112_L3_LAB_inside-1: a private client
curl -v --max-time 5 http://112.0.0.1:8080/
```

Re-key the rule on the address being published instead of the interface the request arrived on. Rules are deleted by handle, and `nft -a` is what prints handles:

```
# 112_L3_LAB_router
nft -a list chain ip nat prerouting          # read the handle on the 'dnat to' line
nft delete rule ip nat prerouting handle <the handle you just read>
nft add rule ip nat prerouting ip daddr 112.0.0.1 tcp dport 8080 dnat to \
    192.168.10.20:80
conntrack -F
```

```
# 112_L3_LAB_inside-1
curl -v --max-time 5 http://112.0.0.1:8080/
```

The failure changed. Watch the client's own interface while the fetch runs, which shows both halves of the exchange in one capture. Start the capture in a second shell on `inside-1`:

```
# 112_L3_LAB_inside-1: second shell
tcpdump -n -i 112-S1 tcp
```

Definition: Hairpinning

A packet entering a router on one interface and being forwarded back out of that same interface. It happens whenever a machine reaches its own site's published address: the destination rewrite sends the packet to a server on the segment it just came from. The name is also used for the extra source NAT that makes such a flow work, which some vendors call NAT loopback.

Question 14. From your capture, give the destination address and port of the packet `inside-1` sends, and the source address and port of the packet that comes back to it. Then explain why `inside-1` does not treat the second packet as the reply to the first.

Hairpinning is what the router is now doing, and the missing piece is that the private web server is replying straight to `inside-1` rather than through the router. It replies straight there because the request it received still carries `inside-1`'s own address as its source, and the two are one hop apart. Change that source, and the reply has to travel back the way the request came:

```
# 112_L3_LAB_router
nft add rule ip nat postrouting iifname "inside" oifname "inside" masquerade
conntrack -F
```

Both interface conditions are doing work. `iifname "inside"` with `oifname "inside"` selects packets that arrived on the private segment and are being sent back out to it, which in this

topology only ever happens to a flow the destination rule turned around: any other pair of private addresses is one hop apart and never reaches the router.

```
# 112_L3_LAB_inside-1
curl http://112.0.0.1:8080/
curl http://112.0.0.1:8080/whoami.cgi
```

Question 15. `/whoami.cgi` through the published address now returns `192.168.10.1` to `inside-1`. Explain how replacing the source address is what gets the reply back to the client, naming the routing decision the private web server makes differently as a result.

Question 16. A request `inside-1` sends straight to `192.168.10.20` still shows `192.168.10.11` at the server, while the same request through the published address shows `192.168.10.1`. Check both, then say what the private web server loses about requests that arrive the second way, and name one thing the site could do instead of this rule so that private clients reach the service without their address being replaced.

Checks: what "done" means

Run **Status** in the TUI, or `scripts/status.sh`. It makes three requests and scores each one twice, on whether the marker of the intended service came back and on which source address that service saw:

- `inside-1` to `http://112.0.0.10/`, seen by the outside server as `112.0.0.1`.
- `outside` to `http://112.0.0.1:8080/`, seen by the private server as `112.0.0.10`.
- `inside-1` to `http://112.0.0.1:8080/`, seen by the private server as `192.168.10.1`.

Six checks pass when all three work and each is seen with the right address. Status also prints the ruleset and the connection-tracking table, which is where to look when a request arrives and the address is wrong. If a rule you are sure is correct appears to do nothing, run `conntrack -F` on the router first: a connection tracked before the rule existed keeps crossing untranslated until its entry expires.

Safety note

Everything here runs on one isolated Open vSwitch bridge and one point-to-point link between containers this lab created. Neither segment has a route off the host, `112.0.0.0/24` is reachable only from inside the lab, and every rule you write lives in the router container's own network namespace and is deleted with it. Nothing in the lab needs `sudo` or `root` on the machine you are sitting at.

Teardown

Remove the lab when you are finished: pick **Teardown** in the TUI, or run `scripts/teardown.sh`.